Interactive 3D Fluid Jet Painting

Sangwon Lee* Sven C. Olsen† Bruce Gooch‡

Department of Electrical Engineering and Computer Science
Northwestern University



Figure 1: *A painting created using our system.*

Abstract

We present an interactive system which allows users to create abstract paintings in the style of Jackson Pollock using three dimensional viscous fluid jets. Pollock's paintings were created by using streams of household paint to make guided, semi-random patterns on his canvas. Our fluid jet model consists of two coupled simulations: a Navier-Stokes solver for an axis-symmetric fluid column and a linked-mass system for tracking the three dimensional motion of the jet's axis line. The paint trails left by the jets are represented using implicit surfaces. Our system also includes an algorithm for generating the splatter patterns created by the impacts of a high-speed fluid drops. We allow users to analyze the fractal properties of the images they create, comparing them to those known to exist in Pollock's own paintings.

CR Categories: I.3.4 [Computer Graphics]: Graphics Utilities—Paint systems; I.3.7 [Computer Graphics]: Three-Dimensional Graphics and Realism—Animation

Keywords: Jackson Pollock, Non-photorealistic rendering, 3D painting system, Fluid jet simulation

1 Introduction

In the 1940s and 50s, Jackson Pollock developed a style of painting known as Abstract Expressionism. He rolled a large canvas across the floor of his barn onto which he dripped, drizzled, and poured household paint. His technique generated a sensation among critics accustomed to traditional brush strokes and raised great debates about the work's aesthetic significance.

With his rather simple painting technique, he managed to create numerous paintings with widely differing visual styles. Common to all of these paintings, however, is Pollock's unique usage of fluid trails with varying widths. The formulation of these trails depends on the interplay between the artist's stroke motion and dynamics of thin, viscous fluid. In order to simulate this interaction, it is critical to have an interface and real-time fluid model that considers all influencing factors; namely, velocity, gravity, viscosity, and surface tension. However, since current 3D fluid jet simulation techniques proved too expensive to be used in our real-time application, we have developed a stable fluid jet model which couples two different fluid types with reduced dimensions. This ensures interactive simulation while preserving characteristic fluid behaviors shown in Pollock's paintings.

A recent mathematical analysis indicates that the fluid jet patterns of Pollock's painting may be related to fractal structure [Taylor et al. 1999]. The thrown paint allows for self-similar patterns which contribute to the aesthetic quality of his work. We have therefore included an evaluation tool which calculates the fractal dimensions of a user's painting. Unlike real-world paintings, our digital system makes users aware of fractal properties interactively, and can compare fractal characteristics of each individual's work easily.

2 Motivation

In the non-photorealistic rendering community, there has been considerable interest in using physically-based simulations to create digital artworks which mimic traditional painting styles [Baxter et al. 2004; Curtis et al. 1997]. Previous research has focused on the motion of paint after it has come in contact with the canvas. In the case of Pollock's paintings, most of the dynamics of the paint takes place while it is falling through the air, making a 3D simulation necessary.

Fluid simulation has received considerable attention in computer graphics, and there are several advanced 3D Eulerian and semi-Lagrangian solvers designed for use in production environments

*e-mail: s-lee21@northwestern.edu
†e-mail:sven2718@verizon.net
‡e-mail:bgooch@cs.northwestern.edu

Copyright © 2006 by the Association for Computing Machinery, Inc.
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions Dept, ACM Inc., fax +1 (212) 869-0481 or e-mail permissions@acm.org.
NPAR 2006, Annecy, France, 05–07 June 2006.
© 2006 ACM 1-59593-357-3/06/0006 $5.00

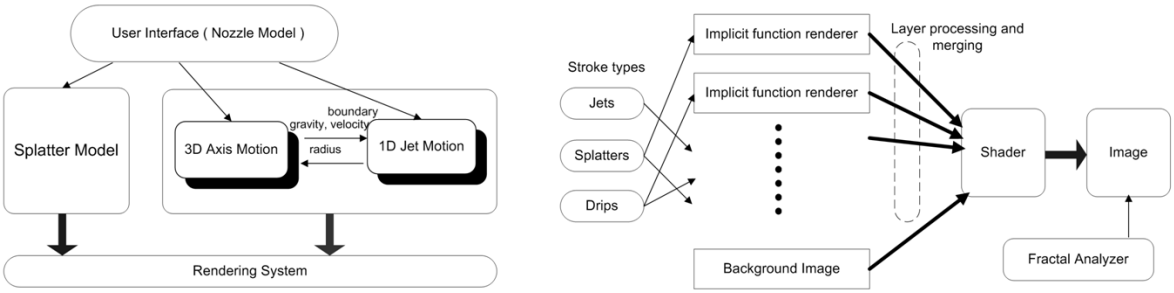


Figure 2: *A diagram of the system detailing the fluid jet and splatter modeling (Left) as well as the rendering components (Right).*

[Enright et al. 2002; Carlson et al. 2004]. However, Simulating Pollock's paint requires modeling a thin fluid jet, and to do this using a Eulerian technique would be difficult, requiring a very fine grid discretization [Hong and Kim 2005]. Recently, adaptive meshing methods have been incorporated into the semi-Lagrangian schemes popular in the animation community, making such solvers much better suited to handling thin fluid streams [Losasso et al. 2004]. While these systems can produce good results, they are still far too slow for an interactive system. Particle-based approaches [Müller et al. 2003] are designed for use in interactive applications, but the computational costs are still high, demanding expensive operations over sets of several thousand particles, and the resulting simulations are relatively inaccurate. It has also proven difficult to improve on the efficiency of particle based approaches by exploiting the parallelism available on GPUs [Amada et al. 2004]. For these reasons, reduced-dimension approximations of 3D fluid motion, such as [Kass and Miller 1990] continue to be popular in interactive applications. The fluid jet model used in our painting system is a reduced-dimension simulation, and, unlike general 3D Eulerian or Lagrangian solvers, it has an almost negligible computational cost.

Fluid jet modeling has several important industrial applications, such as designing inkjet printers or other devices that rely on fluid sprays. Thus, the mechanical engineering literature contains a large body of work on fluid jet modeling. Some of this work uses one-dimensional simulations [Eggers and Dupont 1994], but there the demands for physical accuracy are extreme and the methods that have been developed are slow and unstable. Our model, which emphasizes speed, stability, and flexibility, is a new approach to the old problem of fluid jet simulation.

In the case of Pollock's paint, the paint settles very quickly after hitting the canvas surface, so the characteristic paint patterns are formed in the first instant after impact. Unfortunately, there are currently no models which can effectively simulate the impact-dynamics of high velocity fluid particles. However, there have been some statistical studies of drop-solid collisions [Marmanis and Thoroddsen 1996], and we draw on this work in our own painting system.

3 Fluid Jet Modeling

Our fluid jet modeling system has two components: a one dimensional model used to determine the width of the jet, and 3d model that tracks the motion of the jet centerline. Each model influences the other, and together they form a convincing approximation of fluid jet motion which is both stable and computationally inexpensive. When the jet hits the canvas it leaves behind a smooth trail of paint, which is calculated and maintained using implicit surfaces.

3.1 One Dimensional Fluid Jet Model

For our one dimensional model we assume that the jet centerline is fixed in space and that the jet is axis-symmetric. We then apply the Navier-Stokes equation in cylindrical coordinates. With the continuity condition, the equation is:

$$\partial_t v_r + v_r \partial_r v_r + v_z \partial_z v_r = -\frac{\partial_r p}{\rho} + \nu(\partial_r^2 v_r + \partial_z^2 v_r + \frac{\partial_r v_r}{r} - \frac{v_r}{r^2}) \quad (1)$$

$$\partial_t v_z + v_r \partial_r v_z + v_z \partial_z v_z = -\frac{\partial_z p}{\rho} + \nu(\partial_r^2 v_z + \partial_z^2 v_z + \frac{\partial_r v_z}{r}) + g \quad (2)$$

$$\partial_r v_r + \partial_z v_z + \frac{v_r}{r} = 0 \quad (3)$$

where ν is kinematic viscosity, ρ is density, r is the radius of the jet, p is pressure, v_r is the velocity in the radial direction, v_z is the velocity along the axis, and g is the strength of gravity along the positive z axis. (Where ∂_x represents differentiation with respect to variable x.)

These equations are in 2-dimensional space. Following the analysis of [Eggers and Dupont 1994], we reduce the equations to one dimension by assuming the fluid column is thin relative to its length. The velocity and pressure terms are expanded in a Taylor series with respect to r:

$$v_z(z,r) = v_0 + v_2 r^2 + \ldots \quad (4)$$

$$v_r(z,r) = -\frac{1}{2} v_0' r - \frac{1}{4} v_2' r^3 - \ldots \quad (5)$$

$$p = p_0 + p_2 r^2 + \ldots \quad (6)$$

Equation (5) is chosen to satisfy equation (3) and (4). We substitute these equations for v_z and v_r in (1)-(3). By choosing only the lowest order of r and applying an additional force balance condition on the fluid surface, we get

$$\partial_t v = -v v_z - \frac{p_z}{\rho} + 3\nu\left[\frac{2h_z v_z}{h} + v_{zz}\right] + g \quad (7)$$

$$p = \gamma\left[\frac{1}{h(1+h_z^2)^{\frac{1}{2}}} - \frac{h_{zz}}{(1+h_z^2)^{\frac{3}{2}}}\right] \quad (8)$$

and

$$\partial_t h = -v h_z - \frac{1}{2} v_z h \quad (9)$$

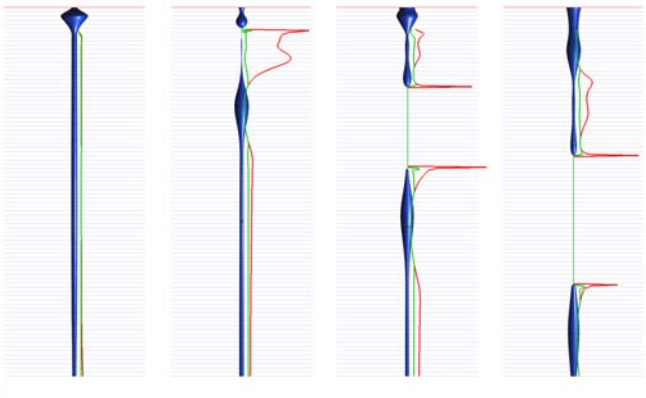


Figure 3: *Profile of the curvatures of a breaking jet. The surface tension value of the 1D jet extends to infinity near the breaking point (red line). We address this by assuming symmetry of curvature when the jet thickness is below a given threshold (green line).*

where h is a distance from the column axis to the jet surface and subscript z refers to differentiation with respect to z. Note that, the equation is in 1-dimensional space and all spatial derivatives are in the z direction. This is a direct consequence of (4) and (5) where the radial and longitudinal components of velocity have both been expanded with respect to a single variable, r.

3.1.1 Numerical Method

Equations (7)-(9) have been used to simulate both dripping and jet streams [Eggers and Dupont 1994; Brenner et al. 1997]. However the numerical methods which have been applied suffer from problems that prohibit their use in graphics applications. The solution method used in [Eggers and Dupont 1994] cannot model a breaking jet, because the simulation diverges near the pinch-off region, while the method of [Brenner et al. 1997] is exceedingly computationally expensive, because of the explicit time integration scheme.

Our solver uses a Preissmann numerical scheme with Newtonian iterations – similar to the method used to solve the equations for shallow water with a free-slip boundary condition in [Abbott and Basco 1989]. We discretize the motion equation (7) as follows:

$$\partial_t v = (1-\psi)\left(\frac{v_i^{t+1}-v_i^t}{dt}\right)+\psi\left(\frac{v_{i+1}^{t+1}-v_{i+1}^t}{dt}\right) \quad (10)$$

$$v_z = (1-\theta)\left(\frac{v_{i+1}^t-v_i^t}{dz}\right)+\theta\left(\frac{v_{i+1}^{t+1}-v_i^{t+1}}{dz}\right) \quad (11)$$

$$h_z = (1-\theta)\left(\frac{h_{i+1}^t-h_i^t}{dz}\right)+\theta\left(\frac{h_{i+1}^p-h_i^p}{dz}\right) \quad (12)$$

$$v_{zz} = \frac{(1-\theta)}{dz}\left[\frac{v_{i+2}^t-v_i^t}{2dz}-\frac{v_{i+1}^t-v_{i-1}^t}{2dz}\right] + \frac{\theta}{dz}\left[\frac{v_{i+2}^p-v_i^p}{2dz}-\frac{v_{i+1}^p-v_{i-1}^p}{2dz}\right] \quad (13)$$

where superscript t refers to time and subscript i refers to spatial position. Superscript p is the linear prediction in time $t+1$ using values in time t and $t-1$. ψ and θ are values between 0 and 1. We can control the stability of the simulation by changing the ψ value: $\psi = 1$ implies a fully implicit scheme, and $\psi = 0$ implies explicit integration. We discretize equation (9) in a similar way except that we exchange h and v in (10)-(12). All spatial differentiation in (8) is done in the current time t.

The nonlinearity issue still remains. The v and h terms in (7) and (9) make equations nonlinear. We linearize by evaluating v and h with all known values at time t [Abbott and Basco 1989]. One estimate for u and v is

$$v^* = \frac{v_i^t+v_{i+1}^t+v_i^p+v_{i+1}^p}{4}$$

$$h^* = \frac{h_i^t+h_{i+1}^t+h_i^p+h_{i+1}^p}{4}$$

Thus (7)-(9) becomes a set of linear equations with unknown terms v_i^{t+1}, v_{i+1}^{t+1}, h_i^{t+1} and h_{i+1}^{t+1}. We vary the boundary conditions h_0 and v_0 according to user input.

We solve this system by using a pair of Newtonian iterations to calculate v and h. We found that in our case, a single Newton iteration is generally sufficient, [Abbott and Basco 1989] notes that this is often the case when Newton iterations are applied to fluid modeling problems. The complete algorithm as it is applied each time step is illustrated in the following pseudo code.

```
UPDATE_1D_JET
    v^p ← 2v - v_old, h^p ← 2h - h_old
    h* ← AVG(h,h^p)
    FOR k = 0 TO kmax
        v* ←AVG(v,v^p)
        new_v ← SOLVE_V_LINEAR_EQNS(v,v*,v^p,h,h*,h^p)
        v^p ← new_v
    END_FOR
    old_v ← v, v ← v^p
    v* ← AVG(v,v^p)
    FOR k = 0 TO kmax
        h* ←AVG(h,h^p)
        new_h ← SOLVE_H_LINEAR_EQNS(v,v*,v^p,h,h*,h^p)
        h^p ← new_h
    END_FOR
    old_h ← h, h ← h^p
```

A numerical difficulty occurs when evaluating equation (8). When the jet breaks and h becomes zero, the right hand side of the equation (8) approaches infinity, due to the presence of h in the denominator. This problem exists because we are calculating the curvature of a three dimensional object approximated using a one dimensional discretization. To overcome this problem, we consider the case where h is small, and rewrite the equations such that z is expressed as a function of h.

$$z = z0 - \alpha h^2 + \ldots \quad (14)$$

We expand z using a Taylor series with respect to h and take the two lowest order terms. By differentiating both sides we get:

$$h_z = -\frac{1}{2\alpha h}, \qquad h_{zz} = -\frac{1}{4\alpha^2 h^3}$$

Then terms on the right hand side of (8) become

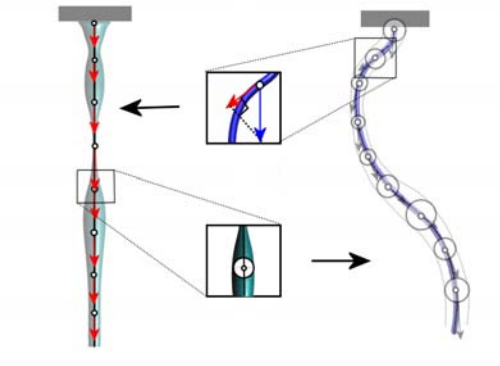


Figure 4: *Coupling of fluid jet models. The tangential component of the gravitational force relative to the 3D axis provides the acceleration used in the 1D jet model (top). Updated jet thickness from 1D jet model becomes the radius distribution along the 3D axis line (bottom).*

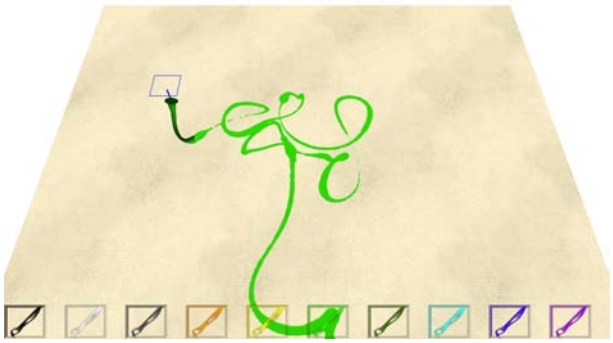


Figure 5: *Perspective view of a 3D jet with paint trails.*

$$\frac{1}{h(1+h_z^2)^{\frac{1}{2}}} = \frac{2\alpha}{(4\alpha^2h^2+1)^{\frac{1}{2}}} \approx 2\alpha \quad (15)$$

$$-\frac{h_{zz}}{(1+h_z^2)^{\frac{3}{2}}} = \frac{2\alpha}{(4\alpha^2h^2+1)^{\frac{3}{2}}} \approx 2\alpha \quad (16)$$

Notice these terms converge to the same value when h approaches zero. Making this symmetry of curvature assumption when the fluid jet breaks allows us to use a numerical value of (15) in place of (16) when h is below a threshold. Figure 3 compares this technique for calculating curvature with the curvature calculated by applying (8) naively.

3.2 3D axis motion

The axis of the fluid jet in (7)-(9) is a straight line. To model the free motion of a jet in 3D space, we discritize the jet centerline in 3D, and apply fluid forces between neighboring points. Neighboring points exchange three types of forces; pressure forces, viscous forces, and surface tension. These forces deform the axis line to form a free curve. The partial differential equation is:

$$\partial_t u = -\frac{\partial_l p}{\rho} + \nu\partial_l^2 u + \frac{2\pi\sigma h}{\rho} + g \quad (17)$$

where ∂_l refers to differentiation along the axis line. Note that because we are using a Lagrangian scheme there is no advection term $u\partial_l u$. We solve these equations using an explicit method both in time and space.

$$\frac{u_i^{t+1}-u_i^t}{dt} = -\frac{p_{i+1}^t-p_{i-1}^t}{2\rho dz} + \nu\frac{u_{i+1}^t-2u_i^t+u_{i-1}^t}{dz^2} + \frac{2\pi\sigma h_i^t}{\rho} + g \quad (18)$$

At each time step, we compute a new discretization along the jet axis line, as well as a new velocity for each point which we use to update its location. Values of pressure p and radius h are given by equations (8)-(9) in 1D jet model.

Rather than using a constant value of g in the 1D solver, we use the component of gravity tangential to the 3D centerline. This is because the apparent gravity at any point in the 1D simulation varies depending on the slope of the jet axis. As the jet becomes increasingly horizontal, the fluid modeled by the 1D simulation experiences less acceleration along the centerline. These variations in g imply that the two models are mutually dependent (Fig. 4). This interdependence has proven essential to producing convincing jet simulations.

However, the gravity projection complicates the 1D solver. When the force of gravity is in the same direction as the fluid flow, we can solve the linear system implied by the simplified Navier-Stokes equations quickly. The solver can start from the boundary point and update values by sweeping down the jet. However, if the force of gravity were to imply an acceleration against the flow of the jet, this fast solution method would fail, as information would need to start traveling up the length of the jet. Because the upward flow case occurs very rarely in our painting application, rather than using a less efficient 1D solver, we instead constrain the projected gravity to be below a given minimum.

3.3 Creating the Paint Trail

In the course of our research, we experimented with modeling the motion of paint on the canvas using a variety of 2D fluid simulations. However, all methods proved ill-suited to the task of modeling the paint trails left by our fluid jets. The problem is particularly difficult for a number of reasons. As we are modeling thin lines of paint over a large area, we need to keep very high resolution data, and this constraint alone makes it difficult to run any grid-based simulation at interactive speeds. While explicit simulations are unstable, semi-Lagrangian methods are not mass conserving, a deficiency which is highlighted when they are applied to the task of diffusing point density sources. The point density case arises often in our own system.

However, on the canvas, the fluid effects present in the paint applied using Pollock's dripping techniques are quite small. Indeed, the only noticeable fluid behavior is the rapid diffusion of paint which occurs immediately after it impacts the canvas, acting to smooth the edges of the paint trails. A very similar smoothing effect can be achieved by using an implicit surface to determine the boundaries of the painted area.

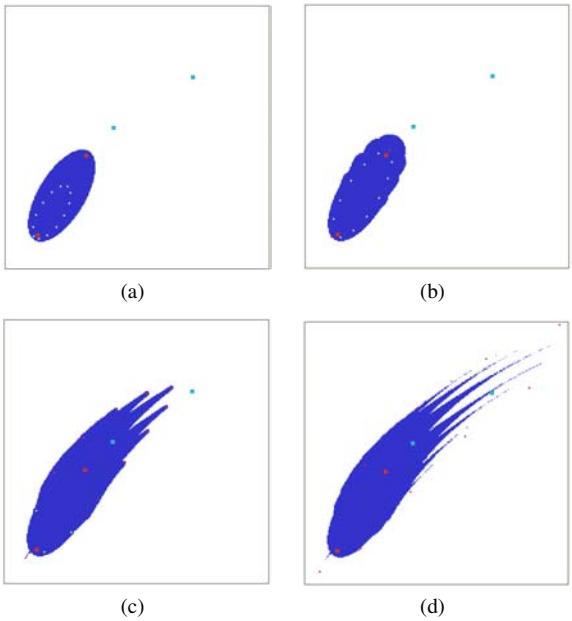


Figure 6: *Splat generation. Particles are created within an ellipse and follow a curved path given by the user. Each particle leaves smooth "fingers" coming out of the ellipse.*

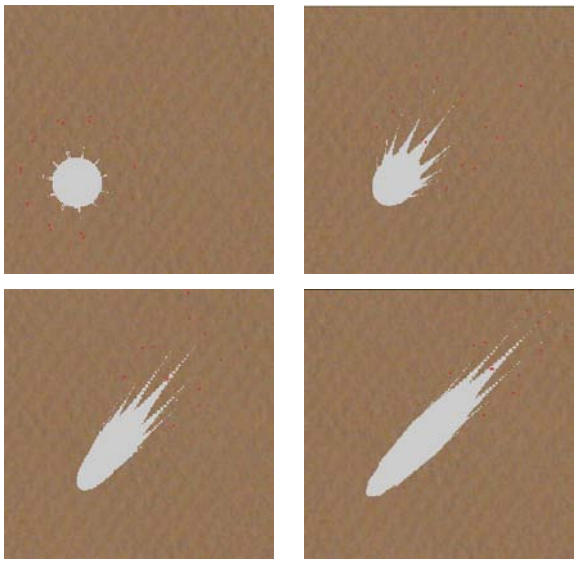


Figure 7: *Splats with different impact velocity.*

Specifically, we define an isofunction $F(x)$, where

$$F(x) \quad = \quad \sum_i \phi_i(||x - p_i||) \tag{19}$$

And $F(x) > \lambda$ implies that a given location is covered in paint. In practice, we have achieved good results using $\lambda = 0.2$, and $\phi_i(r) = \frac{1}{(rs)^3}$, where s varies linearly with the radius of the jet at impact point i. Though in most cases, the fluid jet should leave a continuous line on the canvas, registering distinct impact events is usually sufficient, as the isosurface will contain all intermediate points. In the few cases where the resulting isosurface is discontinuous, the jet is moving sufficiently fast that the continuity assumption of our solver is no longer valid, and thus introducing breaks in the paint trail makes the model appear more realistic than it is.

While this method creates very smooth paint trails, it does so at the cost of any height information. This is acceptable given that we are only trying to model very thin paint, such as the diluted house paint used by Pollock.

Because $\phi_i(r)$ and p_i do not change over time, we can keep track of $F(x)$ by accumulating the results of each new ϕ_i to a high resolution texture the moment that each impact event is registered.

4 Splattering

While thin fluid stream effects are perhaps the most famous of Pollock's techniques, they are by no means the only elements of his paintings. Our system includes other methods for adding Pollock-like elements to a virtual drip painting. The simplest of these is "dripping" command which adds a random distribution of circular ϕ_i's to the current isofunction. Somewhat more sophisticated is a model that allows users to create splattering effects such as those resulting from high velocity fluid particle impacts.

Modeling the spatter created by the high-velocity collision of a fluid drop on a surface is extremely difficult. However, we can use a statistical model to predict the number of "fingers" in a splatter. Experimental data suggests this relation can be modeled by the following relation:

$$N_{fingers} \quad = \quad k \left[\frac{U}{\sqrt{\nu}} \left(\frac{\pi^2 \rho D^3}{16\sigma} \right)^{\frac{1}{4}} \right]^{\frac{3}{4}} \tag{20}$$

where U is the impact velocity and D is the drop diameter, [Marmanis and Thoroddsen 1996].

In order to render splats with the predicted fingered structure, we assign a moving particle to each finger. We first draw an ellipse where the paint impacts the surface, as shown in figure 6(a). The locations of the two focus points are inferred from user input, and are a function of the horizontal component of the drop impact velocity. To determine the initial position of the particles, we first place them uniformly around the edge of the ellipse and contract toward one focal point. Initial velocity is proportional to the distance to the selected focal point. See figure 6(b). We rotate the direction of velocity by a fixed angle whenever the particle diverts from the desired path. To define the width of the finger, we associate each particle with an isofunction as in (19). We pick a function that varies polynomially with velocity, so that once the particle escapes the ellipse, friction and drag forces decrease not only its velocity but also the radius of the implied isosurface. Figure 6(c) and 6(d) shows a splat generated with this technique.

5 Interaction

My painting does not come from the easel ... On the floor I am more at ease. I feel nearer, more a part of the painting, since this way I can walk around it, work from the four sides and literally be in *the*



Figure 8: *User interaction with Diamond-Touch Table.*

painting. This is akin to the method of the Indian sand painters of the West. [Landau 1989]

The Diamond-Touch Table (tm) developed at Mitsubishi Electric Research Laboratories (MERL) provides a large projection table which users can touch in order to drive programs. We have implemented a painting interface using the Diamond-Touch device, allowing experimenters and artists to get inside their painting. The user interface is simple, potential abstract expressionists touch a regions of the screen to create paint bucket objects, which can then be directed around the canvas. We also have a desktop version of the code, in which additional key commands allow for zooming, cropping, and control over the paint simulation variables.

Directly connecting the reported touch positions to the paint simulation is impractical because of occasional inaccuracies in the data. Therefore, we instead use an interface which restricts user's control of the simulations, but guarantees stability. The user's touch locations exert attractive forces on paint jet nozzles, modeled as rigid pendulums with 2 degrees of freedom.

5.1 Implementation

Each active paint layer has an associated patch of texture memory, which is used to store the paint's isofunction and a copy of the isosurface. Each rendering cycle, the isosurfaces are sent to a fragment program responsible for turning them into a final rendering. This program interprets the textures sent to it as paint height-fields, colors them according to the properties of the associated paint, performs lighting calculations if desired, and superimposed the result onto a background painting to form the completed rendering. In order to avoid pixelization, and to give the lighting calculations a little more data to work with, we smooth the isosurfaces using a Gaussian filter before submitting them to the mixing program. We do not model pigment mixing, a choice in keeping with the techniques of Jackson Pollock - who generally only work with one wet layer of paint at a time. When a user finishes with a paint layer, it is committed to the background layer. As the layer mixing program can only handle a limited number of simultaneous layers (determined by the multitexturing capabilities of the video card), it is often necessary to commit one paint layer in order to make room for another.

Each time a paint jet registers an impact event with the canvas, we need to accumulate the implied ϕ_i to the layer's isofunction texture. We have two methods for doing this. By applying a fragment program to a 32bit floating point isofunction texture, we can generate high precision results at the cost of one GPU render per ϕ_i. A faster, less precise method is to pre-generate a collection of 8bit ϕ textures, and use OpenGL's blending routines to render the one which best approximates our current ϕ_i to the layer's isofunction. The fragment program based methods are slower, but result in higher quality paint renderings.

We have implemented our system on an Althon64 3.2ghz box with a NVIDIA Geforce GT6800.

6 Fractal Analysis

Cropped Pollock paintings tend to have fractal dimensions similar to that of the complete painting [Taylor et al. 1999]. Our painting system includes a fractal dimension calculator as an indicator of self similar patterns. To measure the fractal dimension we use the well established 'box counting' method. In this algorithm, an image is covered with multiple grids, where each cell is a square with length L_i. For each grid, the algorithm finds $Num(L_i)$, the number of cells that contain any paint. In our implementation, the largest L_i is half the length of the shorter side of the image, and the smallest is 2 pixels. The graph of $(log(L_i), -log(Num(L_i))$ yields a roughly linear relation with a slope that determines the fractal dimension of the image. If the data forms a straight line, the painting is fractal.

7 Conclusion and Future Work

As Pollock's painting style matured, the fractal dimension of his images increased. This may suggest correlations between the statistical properties of some abstract art and its aesthetic value. While computers can calculate these properties explicitly, humans may only be able to recognize them subconsciously. We have built a system in which users can create Pollock-like images and analyze their fractal properties in real time; thus, users can create abstract art with an awareness of it's statistical qualities.

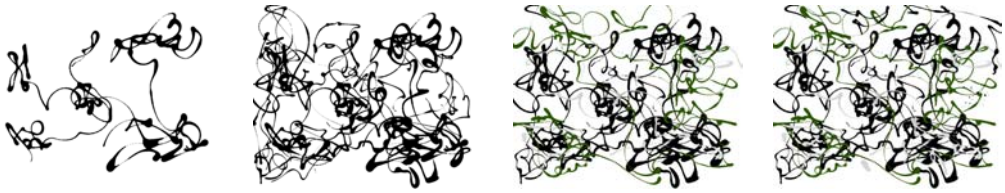
Our digital system provides an interesting environment for further investigation in human-computer interaction. An artificial agent can be a virtual artist, and automatically create different Pollock-style paintings given various aesthetic parameters. The final painting can be compared with others generated by users. Finding characteristics of each user's painting preference will also be an interesting subject for future research. We believe our Pollock-style painting interface may enable the quantification of essential aesthetic elements.

References

ABBOTT, M., AND BASCO, D. 1989. *Computational fluid dynamics: An introduction for engineers*. Wiley.

AMADA, T., IMURA, M., YASUMURO, Y., MANABE, Y., AND CHIHARA, K. 2004. Particle-based fluid simulation on gpu. In *2004 ACM Workshop on General-Purpose Computing on Graphics Processors*, http://www.cs.unc.edu/Events/Conferences/GP2/program.shtml.

BAXTER, W. V., WENDT, J., AND LIN, M. C. 2004. IMPaSTo: A realistic model for paint. In *Proceedings of the 3rd International Symposium on Non-Photorealistic Animation and Rendering*, 45–56.



(a) 1.513 (b) 1.726 (c) 1.741 (d) 1.761

Figure 9: *Change of fractal dimension of paintings. A typical late painting by Jackson Pollock has a value around 1.7.*



Figure 10: *A painting sequence. The choice of colors was inspired by Jackson Pollock's Number 13A, 1948.*

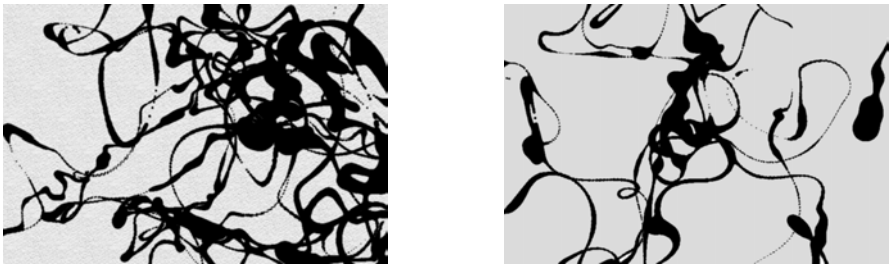


Figure 11: *Detailed views of our strokes. The sparse, two color style is similar to several of Pollock's later works.*



(a) *A result inspired by Autumn Rhythm: Number 30, 1950.* (b) *A result inspired by Number 23, 1948.*

Figure 12: *Reproducing Jackson Pollock's paintings.*

BRENNER, M. P., EGGERS, J., JOSEPH, K., AND SHI., S. R. N. X. D. 1997. Breakdown of scaling in droplet fission at high reynolds number. *Phys. Fluids 9*, 1573–1590.

CARLSON, M., MUCHA, P. J., AND TURK, G. 2004. Rigid fluid: animating the interplay between rigid bodies and fluid. *ACM Trans. Graph. 23*, 3, 377–384.

CURTIS, C. J., ANDERSON, S. E., SEIMS, J. E., FLEISCHER, K. W., AND SALESIN, D. H. 1997. Computer-generated watercolor. In *Proceedings of SIGGRAPH 97*, Computer Graphics Proceedings, Annual Conference Series, 421–430.

EGGERS, J., AND DUPONT, T. D. 1994. Drop formation in a one-dimensional approximation of the navier-stokes equation. *J. Fluid Mech. 262*, 205–221.

ENRIGHT, D., MARSCHNER, S., AND FEDKIW, R. 2002. Animation and rendering of complex water surfaces. In *SIGGRAPH '02: Proceedings of the 29th annual conference on Computer graphics and interactive techniques*, ACM Press, 736–744.

FOSTER, N., AND METAXAS, D. 1996. Realistic animation of liquids. *Graphical Models and Image Processing 58*, 5 (Sept.), 471–483.

HONG, J.-M., AND KIM, C.-H. 2003. Animation of bubbles in liquid . *Comput. Graph. Forum 22*, 3, 253–262.

HONG, J.-M., AND KIM, C.-H. 2005. Discontinuous fluids. *ACM Trans. Graph. 24*, 3, 915–920.

KASS, M., AND MILLER, G. 1990. Rapid, stable fluid dynamics for computer graphics. *SIGGRAPH Comput. Graph. 24*, 4, 49–57.

LANDAU, E. G. 1989. *Jackson Pollock*.

LOSASSO, F., GIBOU, F., AND FEDKIW, R. 2004. Simulating water and smoke with an octree data structure. *ACM Trans. Graph. 23*, 3, 457–462.

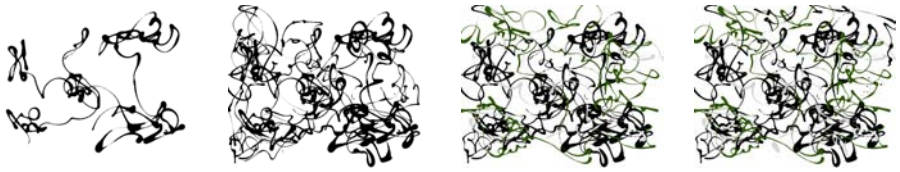
MARMANIS, H., AND THORODDSEN, S. T. 1996. Scaling of the fingering pattern of an impacting drop. *Phys. Fluids 8*, 1344–1346.

MÜLLER, M., CHARYPAR, D., AND GROSS, M. 2003. Particle-based fluid simulation for interactive applications. In *SCA '03: Proceedings of the 2003 ACM SIGGRAPH/Eurographics Symposium on Computer animation*, Eurographics Association, 154–159.

O'BRIEN, J. F., AND HODGINS, J. K. 1995. Dynamic simulation of splashing fluids. In *Computer Animation '95*, 198—-205.

STAM, J. 1999. Stable fluids. In *Proceedings of SIGGRAPH 99*, Computer Graphics Proceedings, Annual Conference Series, 121–128.

TAYLOR, R. P., MICOLICH, A. P., AND JONAS, D. 1999. Fractal analysis of pollock's drip paintings. *Nature*, 399–422.



(a) 1.513 (b) 1.726 (c) 1.741 (d) 1.761

Figure 10: *Change of fractal dimension of paintings. A typical late painting by Jackson Pollock has a value around 1.7.*



Figure 11: *A painting sequence mimicking Jackson Pollock's Number 13A, 1948.*



(a) *Number 23, 1948, courtesy of tate.org.uk.*

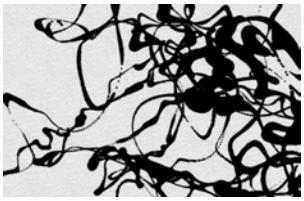


(b) *Our result.*

Figure 12: *Comparision with Jackson Pollock's painting.*



(a) *Number 29, 1950 (Detail), courtesy of National Gallery of Canada.*



(b) *Our result.*



(c) *Our result mimicking Autumn Rhythm: Number 30, 1950.*

Figure 13: *Detailed comparison with Jackson Pollock's paintings.*